



PDF Download
3749566.3749595.pdf
03 January 2026
Total Citations: 0
Total Downloads: 276

 Latest updates: <https://dl.acm.org/doi/10.1145/3749566.3749595>

RESEARCH-ARTICLE

Maliciously Secure Privacy-Preserving Neural Network Inference Framework Based on Function Secret Sharing

MENGBO LI, Southeast University, Nanjing, Jiangsu, China

RUI JIANG, Southeast University, Nanjing, Jiangsu, China

Open Access Support provided by:

Southeast University

Published: 16 May 2025

[Citation in BibTeX format](#)

IoTML 2025: 2025 5th International
Conference on Internet of Things and
Machine Learning
May 16 - 18, 2025
Nanchang, China

Maliciously Secure Privacy-Preserving Neural Network Inference Framework Based on Function Secret Sharing

Mengbo Li

School of Cyber Science and Engineering
Southeast University
Nanjing, Jiangsu, China
mengbo-li@seu.edu.cn

Rui Jiang*

School of Cyber Science and Engineering
Southeast University
Nanjing, Jiangsu, China
R.Jiang@seu.edu.cn

Abstract

With the widespread adoption of machine learning as a service (MLaaS), ensuring the privacy of user data and neural network models has become a critical challenge. Current privacy-preserving neural network inference protocols predominantly follow semi-honest security assumptions, leaving practical and efficient solutions secure against malicious adversaries an open and pressing issue. In this paper, we propose MSPI-FSS, a maliciously secure privacy-preserving neural network inference framework based on function secret sharing (FSS), designed explicitly for secure client-server inference scenarios. To robustly counteract malicious adversaries, we integrate additive secret sharing with SPDZ message authentication code (MAC) to ensure both input privacy and inference result integrity. Furthermore, we design a secure two-party comparison protocol for non-linear layers based on FSS, which significantly reduces communication rounds. Experimental evaluations demonstrate that our proposed MSPI-FSS framework achieves significant performance improvements compared to existing maliciously secure protocols, such as Muse.

CCS Concepts

• **Security and privacy** → Security services; Privacy-preserving protocols; • **Theory of computation** → Theory and algorithms for application domains; Machine learning theory.

Keywords

privacy-preserving machine learning, secure multi-party computation, function secret sharing

ACM Reference Format:

Mengbo Li and Rui Jiang. 2025. Maliciously Secure Privacy-Preserving Neural Network Inference Framework Based on Function Secret Sharing. In *2025 5th International Conference on Internet of Things and Machine Learning (IoTML 2025)*, May 16–18, 2025, Nanchang, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3749566.3749595>

1 Introduction

Deep neural networks have witnessed a significant development in recent years, and various applications based on neural networks

such as medical diagnosis and object recognition have emerged. Machine learning as a service (MLaaS), which aims to provide neural network inference services, has attracted considerable attention. In MLaaS, model owner provides a well-trained neural network, and user utilizes the inference service for data analysis.

Unfortunately, MLaaS systems suffer from serious privacy and security concerns. On the one hand, clients need to send sensitive inputs to the service provider, which could compromise the data privacy of these clients if the provider is untrusted. On the other hand, alternatively, the provider may distribute the proprietary model to clients, but this causes intellectual property violations. Secure multi-party computation (MPC) is used to realize privacy-preserving machine learning (PPML), addressing the dilemma of privacy concerns in ML. In earlier work, SecureML [1] is the first privacy-preserving neural network framework and implementation with secure two-party computation based on secret sharing and garbled circuits. Based on this work, MiniONN [2] proposed a method that accelerated the generation of Beaver triples with added homomorphic encryption. Delphi [3] proposes to reduce the online overhead of performing the inference process by dividing the inference service into an offline phase for preprocessing and an online phase for executing the inference.

The protocol mentioned above only considers semi-honest adversary scenarios, where participants are considered honest-but-curious. However, in real-world scenarios, malicious parties may potentially compromise the privacy of data and the correctness of results. For example, a malicious client could deviate from the protocol to mount model extraction attacks, stealing the server's model with far fewer queries than black-box attacks [4]. A malicious server might send manipulated results or alter the computation to infer the client's input, violating integrity and privacy. These limitations underscore the need for frameworks that achieve both robustness and efficiency under the malicious security model. Muse [5] suggests that malicious clients can extract model parameters in the inference process, and further proposes a two-party inference scheme based on Delphi [3] that can resist malicious clients. Muse protocol is based on SPDZ message authentication code (MAC) [6] to authenticate the results. Some works [7–14] consider privacy protection inference malicious security in multiple participant scenarios, which are different from the two-party inference scenarios we focus on but more relevant to outsourced computing.

Beyond security considerations, practical deployment of privacy-preserving neural networks crucially depends on computational efficiency, particularly at the non-linear layers. Earlier research [3, 5, 13, 15, 16] typically used Garbled Circuit (GC) or bit-wise comparison designs to implement these layers. GC-based methods

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *IoTML 2025, Nanchang, China*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1392-7/2025/05

<https://doi.org/10.1145/3749566.3749595>

[3, 5] although effective for constant-round communications, require extensive communication overhead due to oblivious transfers. Bit-wise comparison approaches [15, 16] can achieve high communication throughput but require frequent interactions between participants. When considering malicious security, communication efficiency is further compromised. Especially in Muse, to combine authenticated secret sharing and garbled circuit, a complex switch protocol is proposed, which affects the efficiency of inference. Fortunately, Boyle et al. [17] proposes a secure two-party computation protocol based on function secret sharing. The online communication rounds of non-linear functions can be reduced to a constant round by a distributed comparison function (DCF). Ryffel et al. [18] propose AriaNN, a privacy-preserving inference framework based on function secret sharing. AriaNN designs a novel DCF gate with reduced key sizes and presents a secure ReLU protocol based on their DCF construction. FastSecNet [19] further minimizes the communication rounds of the ReLU function to a single round but introduces more computational overheads.

In this paper, we propose the Maliciously Secure Privacy-Preserving Neural Network Inference Framework Based on Function Secret Sharing (MSPI-FSS). The key contributions of our MSPI-FSS are as follows:

- 1) We propose a maliciously secure privacy-preserving neural network inference framework. Unlike existing frameworks primarily designed for semi-honest adversarial models, our framework effectively addresses malicious adversaries by integrating additive secret sharing with SPDZ Message Authentication Code (MAC). This combination ensures data and model privacy while safeguarding inference correctness from external adversary and malicious participants.
- 2) we design a novel comparison protocol based on function secret sharing. To overcome communication overhead, we design a secure comparison protocol leveraging FSS that achieves constant-round communication, which significantly reduces overhead without compromising security in the malicious adversary model.
- 3) Experimental evaluations demonstrate that our framework outperforms state-of-the-art maliciously secure inference methods such as Muse and FastSecNet in terms of inference speed and computational efficiency. These results validate the practicality and effectiveness of our approach for real-world applications.

2 Preliminaries

2.1 Authenticated Secret Sharing

We design our protocols using the authenticated additive sharing semantics described in SPDZ_{2k} [6], which use message authentication code (MAC) for authentication.

For a 2-of-2 additive secret sharing, the additive secret share of a number $x \in \mathbb{Z}_N$ is denoted as $[x]$, and each party P_b holds $[x]_b$, with constraint that $x = [x]_0 + [x]_1$. Let s be the statistical security parameter. Let $\alpha \in \mathbb{Z}_{2^s}$ denote a global key that is additively shared among the parties. Let $v \in \mathbb{Z}_{2^k}$ be the secret value to be an authenticated shared. Let $[x] \in \mathbb{Z}_{2^{k+s}}$ denote the additive shares of $x \in \mathbb{Z}_{2^k}$ in a larger ring $\mathbb{Z}_{2^{k+s}}$ and $[M_x]$ denote the additive secret shares of $\alpha x \in \mathbb{Z}_{2^{k+s}}$. We denote authenticated secret share $\langle x \rangle$ as the tuple $([x], [M_x])$.

Notice that the authenticated secret-sharing scheme satisfies the linearity property. The linear combination of secret-shared values can be locally computed by the parties. For given secret-shared value $\langle x_1 \rangle, \dots, \langle x_m \rangle$, parties can compute the linear combination $\langle c_1 x_1 + c_2 x_2 + \dots + c_m x_m \rangle$ non-interactively by locally multiplying and adding the additive shares.

Definition 1 (MACCheck) [6]. Let α be the MAC key additively shared among client and server. Let the client and the server holds authenticated secret-sharing $\langle Y \rangle$, which means the client holds $Y_0 = \{[Y]_0, [M_Y]_0\}$ and the server holds $Y_1 = \{[Y]_1, [M_Y]_1\}$.

Firstly, the client and the server sample a public random vector $\chi \in \mathbb{Z}_{2^n}^t$. Secondly, the client and the server broadcast their shares $[Y]$ and reconstructs $Y = [Y]_0 + [Y]_1$. Then, the client computes $[C]_0 = [M_Y]_0 \cdot \chi - [\alpha]_0(Y \cdot \chi)$ and commits to $[C]_0$, while the server commits to $[C]_1 = [M_Y]_1 \cdot \chi - [\alpha]_1(Y \cdot \chi)$. Finally, client and server open their commitments and verify that $\sum_i^2 [C]_i = 0$. If the check fails, then parties abort.

2.2 Multiplication with Beaver Triple

Beaver triple is adapted for compute the multiplication. A multiplication beaver triple is a triple (a, b, c) , such that $ab = c$. In our two-party setting, we generate the authenticated secret shares beaver triple as $(\langle a \rangle, \langle b \rangle, \langle c \rangle)$. Give two-party authenticated secret sharing $\langle x \rangle$ and $\langle y \rangle$, the multiplication can be computed as following. First, parties should reconstruct the mask value $x - a$ and $y - b$. Then, the result $\langle z \rangle$ can be computed locally. Detailly, one party locally computes $\langle z \rangle_0 = \langle b \rangle_0(x - a) + \langle a \rangle_0(y - b) + \langle c \rangle_0$ with the beaver triple sharing, while the other party locally computes $\langle z \rangle_1 = (x - a)(y - b) + \langle b \rangle_1(x - a) + \langle a \rangle_1(y - b) + \langle c \rangle_1$.

2.3 Function Secret Sharing

Definition 2 (FSS: Syntax) [20]. A (2-party) function secret sharing (FSS) scheme is a pair of algorithms (KeyGen, Eval) with the following syntax.

KeyGen($1^\lambda, f$) is a probabilistic polynomial-time key generation algorithm that takes as input the security parameter λ and a function f , and outputs a pair of keys (k_0, k_1) .

Eval(i, k_i, x) is a probabilistic polynomial-time evaluation algorithm, which inputs party index $b \in \{0, 1\}$, k_b and $x \in \mathbb{Z}_N$, outputs $f_b(x) \in \mathbb{Z}_N$ (the share of $f(x)$).

Definition 3 (Distributed Comparison Function, DCF) [17, 20]. A special interval function $f_{\alpha, \beta}^< : \mathbb{U}_N \rightarrow \mathbb{G}_{out}$, also called comparison function, takes as an input $x \in \mathbb{U}_N$ and outputs β if $x < \alpha$ and 0 otherwise. the corresponding FSS scheme for this function ($Gen_{\alpha, \beta}^<, Eval_{\alpha, \beta}^<$) is called Distributed Comparison Function.

3 Our MSPI-FSS Framework

3.1 System Model

In this paper, we propose a secure two-party neural network framework to enable practical and maliciously secure inference. As illustrated in Figure 1, the system model of our MSPI-FSS framework consists of three different entities which are client, server and semi-honest third party (STP). The STP generates related randomness and distributes them to the client and server in the preprocessing

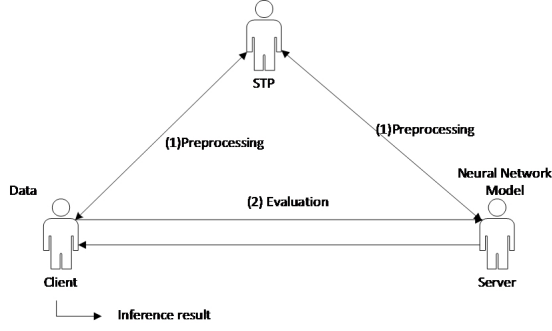


Figure 1: System Model of MSPI-FSS.

phase. The server holds the neural network model with private parameters. The client holds the private query input data. The client and the server interactively perform inference protocols. Finally, the client obtains the neural network inference result while not revealing any information about private input.

3.2 Threat Model

In the MSPI-FSS framework, we consider an adversarial setting involving malicious adversaries and external adversaries. A malicious adversary can arbitrarily deviate from the protocol, including injecting incorrect messages or modifying computations. In our model, both the client and the server are assumed to be malicious and may act dishonestly to compromise the protocol's integrity or privacy.

The semi-honest third-party server (STP) is only involved in the preprocessing phase to generate correlated randomness. It follows the protocol honestly and does not participate in the online inference, ensuring that it learns nothing about the inputs. All communications with the STP are encrypted, preventing any input leakage between the client and server.

Additionally, we account for an external adversary capable of intercepting and tampering with messages exchanged during inference. This adversary can eavesdrop on communications and alter intermediate results to disrupt the correctness of the output.

3.3 Detailed Protocol

3.3.1 Input Masking Protocol. In our MSPI-FSS framework, all secret-shared values are represented using SPDZ-style authenticated additive shares, where each share is accompanied by a message authentication code (MAC) to ensure integrity against malicious adversaries. We design an Input Masking Protocol that securely establishes authenticated shares of private inputs between the client and server, leveraging the assistance of a semi-honest third party (STP) during the preprocessing phase.

As illustrated in Figure 2, the protocol operates between two principal parties P_0 and P_1 , according to the following phases. In the preprocessing phase, P_0 randomly generates a masking parameter r and transmits it to STP. Then, the STP generates and distributes authenticated secret shares $\langle r \rangle_0$ and $\langle r \rangle_1$. In evaluation phase, P_0 samples $[x]_1$ and transmits the masked data $d = r - x$ and $[x]_1$ to P_1 . Then P_0 locally computes $[M_x]_0$ and obtains x_0 , while P_1

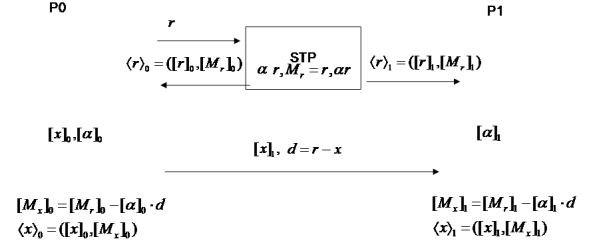


Figure 2: Input Masking Protocol.

locally compute $[M_x]_1$ and obtains x_1 . Let the client be P_0 and the server be P_1 , through the Input Mask Protocol, we obtain the authenticated secret shares $\langle X \rangle$. Similarly, let the server be P_0 and the client be P_1 , we obtain the authenticated secret shares $\langle M \rangle$.

3.3.2 Linear Computation Protocol. For linear computation, we utilize authenticated additive secret sharing combined with Beaver triple-based multiplication to ensure efficient and maliciously secure computation of linear neural network layers, including fully connected layers, convolutional layers, and batch normalization layers.

Fully Connect Layers

The inputs for secure fully connected layer protocol are the input shares X_0 and X_1 , fully connect layers weight parameters shares W_0 and W_1 , fully connect layers biases parameters shares B_0 and B_1 . The client and the server jointly compute based on beaver triples to obtain WX_0 and WX_1 .

The fully connected layer, in essence, performs dot products between the data and the weight parameters, while the convolutional layers require the product of the convolutional kernel parameters and the inputs. Both the fully connect layer and convolution layer can be computed following our secure batch multiplication protocol.

Secure Convolution Layers

At each location, a sum of an element-wise product between the kernel and corresponding data values within the kernel window is computed. More specifically, within each input window, we expand the convolution kernel into a secret sharing vector $\langle X \rangle$, and expand the input window into a secret sharing vector $\langle K \rangle$. Using the multiplication protocol, we can obtain the values for each window.

Batch Normalization Layers

Batch normalization performs element-wise normalization on each neuron's feature. The function can be represented as $\epsilon_1 X + \epsilon_2$. With the input shares X_0 and model parameter shares $\langle \epsilon_1 \rangle$ and $\langle \epsilon_2 \rangle$, the client and the server jointly compute the multiplication with beaver triples to obtain $\epsilon_1 x + \epsilon_2$.

3.3.3 Non-Linear Computation Protocol. For non-linear computation, we introduce a maliciously secure two-party Secure Comparison Protocol based on FSS as the core building block for all non-linear operations, including activation layers and maxpooling layers.

Secure Comparison Protocol

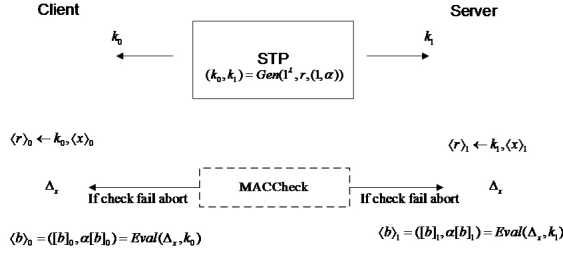


Figure 3: Secure Comparison Protocol.

For non-linear computation, we first designed a secure comparison protocol. We utilize a distributed comparison function from FSS to pre-compute keys, so that at runtime only a single message exchange is needed for ReLU evaluation. As shown in Figure 3, in the preprocessing phase, the STP randomly selects a secret value r . Let $a = r$ and construct $b = (1, \alpha)$, where α serves as the MAC authentication key. The STP then executes the *Gen* algorithm described in Definition 3 to generate correlated cryptographic keys and distribute them securely to both parties. During the evaluation phase, the two parties open and check Δ_x and locally execute the *Eval* protocol. Finally, they obtain authenticated secret shares $\langle b \rangle$.

As described above, our FSS-based comparison protocol achieves constant-round communication—specifically, only two rounds are required to complete a secure comparison. This is a significant improvement over traditional approaches such as those in Cryptflow2 [21] and ABY [16], where secure comparison typically incurs $\log n$ communication rounds. Furthermore, unlike garbled circuit (GC)-based methods, which involve expensive oblivious transfers, our protocol simply involves reconstructing a masked value and exchanging FSS evaluation outputs.

Based on the secure comparison protocol, we construct the secure non-linear layer protocols, including ReLU and maxpooling layer.

Activation Layers

For activation layers, we consider the computation of ReLU function, which is the default and most common activation function in Convolution Neural Network (CNN). ReLU function is defined as $ReLU(x) = \max(x, 0)$. Given secret shares of an input x , secure ReLU protocol is to realize the function. We employ our secure comparison protocol to implement the protocol. For input shares x_0 and x_1 , client and server execute secure comparison protocol to get secret-shared $Sign(x)$. Then, they use comparison protocol to compute the product of $Sign(x)$ and x as $\prod_{Mult}(\langle x \rangle, \langle Sign(x) \rangle)$. Finally, the server gets $\langle Relu(x) \rangle_0$, the client gets $\langle Relu(x) \rangle_1$. The secure ReLU can be generalized to vectors and matrices trivially.

MaxPooling Layers

The maxpooling layer is used to down sample the features by choosing the maximum values within a certain sliding window. The underlying algorithm of maxpooling layer is to compute the maximum value over $d \times d$ elements $x_1, x_2, \dots, x_{d \times d}$. We observe that $\max(x, y) = Relu(y - x) + x$, hence the maxpooling sources form the evaluation of ReLU. The maxpooling can be realized by using a tree-reduction architecture, which recursively

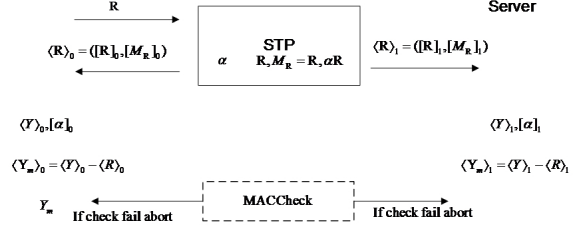


Figure 4: Output and Verification Protocol.

partitions the input into two halves and then compares the elements of each half. Taking $d = 2$ as an example, maxpooling result is $\max(\max(x_0, x_1), \max(x_2, x_3))$.

3.3.4 Output and Verification Protocol. After processing all layers (linear and non-linear), the client and server hold an authenticated secret share of the final inference output. To enable the client to reconstruct the plaintext result Y while preserving confidentiality, we propose the Output and Verification Protocol outlined in Figure 4, which operates as follows: In the preprocessing phase, the client randomly generates a masking parameter R and transmits it to the STP. Then, the STP generates authenticated secret shares $\langle R \rangle_0$ and $\langle R \rangle_1$ and samples a public random vector $\chi \in \mathbb{Z}_{2n}^t$, distributes them securely to the client and the server respectively. In the evaluation phase, both parties collaboratively compute the masked output $[Y_m] = [Y] - [R]$. The client and the server jointly execute *MACCheck* procedure. If the check fails, then abort. Upon successful verification, the client derives locally by removing the mask $Y = Y_m + R$.

4 Security Analysis

4.1 Formal Proof

Theorem 1. Our MSPI-FSS is secure against malicious client and malicious server.

Proof. We construct a probabilistic polynomial time (PPT) simulator *Sim* to generate the transcript of the view of the corrupted client and server. We will show that the real-world distribution is computationally indistinguishable from the simulated distribution by the simulation *Sim* in the ideal world.

Firstly, we construct a *Sim* to generate the view of the corrupted client.

Hyb0: It represents the real-world distribution where the server holds its real model parameters M .

Hyb1: For the Input Mask Protocol, *Sim* sends $[M]_0, D = R - M$ to the client. As $[M]_0$ is a uniformly random value. As the R is a uniformly random value, Hyb1 is computationally indistinguishable from Hyb0.

Hyb2: In this hybrid, for every linear layer, we use the simulator for the Beaver's multiplication procedure. Again, following from the simulation security of this protocol, this hybrid is computationally indistinguishable to Hyb1.

Hyb3: For the non-linear protocol, *Sim* runs the simulators of the FSS protocols. *Sim* open and check Δ_x . Based on the security of the FSS protocols, Hyb3 is computationally indistinguishable from Hyb2.

Hyb4: For the Output and Verification Protocol, the simulator executes *MACCheck* for Y_m . If *MACCheck* abort, then Sim abort. According to Definition 1, only Y_m is open and check, based on the security of MAC, Hyb4 is distributed identically to Hyb3.

Then, we construct Sim to generate the view of the corrupted server.

Hyb0: It represents the real-world distribution where the client holds its real query data X .

Hyb1: For the Input Mask Protocol, Sim sends $[X]_0, D = R - X$ to the server. As $[X]_0$ is uniformly random value. As the R is uniformly random value, Hyb1 is computationally indistinguishable from Hyb0.

Hyb2: In this hybrid, for every linear layer, we use the simulator for the Beaver’s multiplication procedure. Again, following the simulation security of this protocol, this hybrid is computationally indistinguishable from Hyb1.

Hyb3: For the non-linear protocol, Sim runs the simulators of the FSS protocols. Sim open and check Δ_x . Based on the security of the FSS protocols, Hyb3 is computationally indistinguishable from Hyb2.

Hyb4: For the output and check protocol, the simulator executes *MACCheck* for Y_m . If *MACCheck* abort, then Sim abort. According to Definition 1, only Y_m is open and check, based on the security of MAC, Hyb4 is distributed identically to Hyb3.

Theorem 2. Our MSPI-FSS is secure against external adversary.

Proof. To prove the security of the MSPI-FSS protocol against external adversary, we construct a probabilistic polynomial time (PPT) simulator Sim to generate the transcript of the view of the external adversary. We will show that the real-world distribution is computationally indistinguishable from the simulated distribution by the simulation Sim in the ideal world.

Hyb0: It represents the real-world distribution where the client holds its real query data X and the server hold model parameters M .

Hyb1: For the Input Mask Protocol, Sim sends $[X]_0, D = R - X$. As $[X]_0$ is uniformly random value. As the R is uniformly random value, Hyb1 is computationally indistinguishable from Hyb0.

Hyb2: In this hybrid, for every linear layer, we use the simulator for the Beaver’s multiplication procedure. Again, following the simulation security of this protocol, this hybrid is computationally indistinguishable from Hyb1.

Hyb3: For the non-linear protocol, Sim runs the simulators of the FSS protocols. Sim open and check Δ_x . Since the Δ_x is a masked value, Hyb3 is computationally indistinguishable from Hyb2.

Hyb4: For the output and check protocol, the simulator executes *MACCheck* for Y_m . If *MACCheck* abort, then Sim abort. According to Definition 1, only Y_m is open and check, based on the security of MAC, Hyb4 is distributed identically to Hyb3.

4.2 Security Comparison

In this section, we compare the security of our MSPI-FSS with the state-of-the-art secure inference protocols, including Cryptflow2 [21], Ariann [18], FastSecNet [19] and Muse [5].

As shown in Table 1, CryptFlow2, AriaNN and FastSectNet restrict their security guarantees to semi-honest settings, where participants follow the protocol but may attempt to infer private information from intermediate results. Although Muse introduce partial malicious-client resistance, it does not mitigate server-side threats such as inference output tampering. In contrast, MSPI-FSS is against both malicious client and malicious server through robust authentication mechanisms. Furthermore, we consider external adversaries capable of eavesdropping or modifying communications between client and server.

5 Experimental

5.1 Experimental Setting

MSPI-FSS is implemented in Python and Rust, and we base it on the Syft [22] framework to implement our protocol. We set parameters $s = k = 32$, which means the bit length of the fixed-point number is 64. The secure parameters λ is set 32. All the experiments are executed on a server with Intel(R) Core i7-1185G7 and 16GB of RAM running the Ubuntu 20.4 system. We simulate a local-area network (LAN) environment.

MNIST [23] is a dataset for handwritten digit recognition. The training set has 60,000 images and the test set has 10,000 images in 10 classes. Each sample is a single channel (i.e., grayscale) 28×28 image. Our experiments use a 4-layer CNN architecture specified in MiniONN [2].

CIFAR-10 [24] is a dataset with 10 classes. It contains 50,000 training samples and 10,000 test samples. Each sample is a three-channel (i.e., RGB) 32×32 image. Our experiments use a 7-layer CNN architecture specified in MiniONN.

5.2 Comparison with Other Works

We compare the performance of MSPI-FSS with recent private inference framework AriaNN and Muse. To provide a fair and consistent comparison, we re-implemented and evaluated both AriaNN and Muse on the same hardware and experimental setup as MSPI-FSS. Table 2 presents the runtime performance over the MNIST and CIFAR-10 datasets.

The results demonstrate that MSPI-FSS achieves lower communication overhead compared to existing methods. As shown in Table 2, compared to AriaNN, our method introduces authenticated secret sharing to ensure malicious security while maintaining linear time complexity. Notably on the MNIST dataset, MSPI-FSS completes inference within 0.321 seconds while maintaining malicious security, achieving competitive practical performance comparable to AriaNN (0.097s) that only satisfies semi-honest security. This demonstrates our framework successfully bridges the performance gap between semi-honest and malicious-secure paradigms.

When contrasted with the malicious security protocol Muse, our protocol demonstrates significant efficiency. On MNIST/CIFAR10 inference tasks, MSPI-FSS reduces total runtime from Muse’s 14.92/210.49 seconds to 0.321/3.695 seconds. The reason is that our FSS-based comparison protocol eliminates Muse’s dependency on garbled circuit (GC) for non-linear operations, reducing non-linear computation latency through two-round FSS evaluations. These results validate our layered security-efficiency paradigm for privacy-preserving deep learning.

Table 1: Security Comparison.

	Semi-honest Adversary	Malicious Client	Malicious Server	External Adversary
CryptFlow2	✓	×	×	×
Delphi	✓	×	×	×
AriaNN	✓	×	×	×
FastSecNet	✓	×	×	×
Muse	✓	✓	×	×
Our MSPI-FSS	✓	✓	✓	✓

Table 2: Comparison with Prior Frameworks.

Framework	Dataset (model)	Run Time (Sec)		
		Prep.	Eval.	Total
MSPI-FSS	MNIST(CNN-4)	0.235	0.086	0.321
	CIFAR10(CNN-7)	3.144	0.541	3.695
AriaNN	MNIST(CNN-4)	0.061	0.036	0.097
	CIFAR-10(CNN-7)	0.121	0.137	0.258
Muse	MNIST(CNN-4)	14.12	0.80	14.92
	CIFAR-10(CNN-7)	201.12	9.37	210.49

6 Conclusion

In this paper, we propose MSPI-FSS, a maliciously secure privacy-preserving inference framework. Specifically, we introduce a message authentication code in privacy-preserving inference, which protects the protocol against malicious adversaries. By introducing function secret sharing, we reduce inference communication costs. The experimental results demonstrate the practicality of MSPI-FSS and its substantial performance advantage over existing works. Our framework is well-suited for deployment in MLaaS platforms handling sensitive data, such as healthcare diagnostics or financial analytics, where both integrity and confidentiality are critical.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (NO.61372103), Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China and Henan Key Laboratory of Network Cryptography (No.KLCS20240301), and National Engineering Research Center of Classified Protection and Safeguard Technology for Cybersecurity (C23640-XD-07). Rui Jiang is the corresponding author.

References

- [1] MOHASSEL, P. and ZHANG, Y. 2017. SecureML: A system for scalable privacy-preserving machine learning. In *2017 IEEE symposium on security and privacy (SP)* IEEE, 19-38.
- [2] LIU, J., JUUTI, M., LU, Y., and ASOKAN, N. 2017. Oblivious neural network predictions via minionn transformations. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 619-631.
- [3] MISHRA, P., LEHMKUHL, R., SRINIVASAN, A., ZHENG, W., and POPA, R.A. 2020. Delphi: A Cryptographic Inference Service for Neural Networks. In *Proceedings of the 29th USENIX Security Symposium (2020)*, 2505-2522.
- [4] WANG, X., LIU, X., YI, X., YANG, X., and GONDAL, I. 2025. Model Extraction Attacks on Privacy-Preserving Deep Learning Based Medical Services. In *Proceedings of the Web Information Systems Engineering – WISE 2024 (Singapore2025)*, Springer Nature Singapore, 317-332.
- [5] LEHMKUHL, R., MISHRA, P., SRINIVASAN, A., and POPA, R.A. 2021. MUSE: Secure Inference Resilient to Malicious Clients. In *Proceedings of the 30th USENIX Security Symposium (Electr Network, Aug 11-13 2021)*, 2201-2218.
- [6] CRAMER, R., DAMGÅRD, I., ESCUDERO, D., SCHOLL, P., and XING, C. 2018. SPDZ2k: Efficient MPC mod 2^k for Dishonest Majority. In *Proceedings of the 38th Annual International Cryptology Conference (CRYPTO)* (Univ Calif, Santa Barbara, CA2018), 769-798.
- [7] KOTI, N., PATRA, A., RACHURI, R., and SURESH, A. 2022. Tetrad: Actively Secure 4PC for Secure Training and Inference. In *Proceedings of the 29th Annual Network and Distributed System Security Symposium (NDSS)* (San Diego, California, USA2022).
- [8] WAGH, S., TOPLE, S., BENHAMOUDA, F., KUSHILEVITZ, E., MITTAL, P., and RABIN, T. 2021. Falcon: Honest-Majority Maliciously Secure Framework for Private Deep Learning. In *Proceedings of the Privacy Enhancing Technologies (2021)*, 188-208.
- [9] YUAN, B., YANG, S., ZHANG, Y., DING, N., GU, D., and SUN, S.-F. 2024. MD-ML: Super Fast Privacy-Preserving Machine Learning for Malicious Security with a Dishonest Majority. In *Proceedings of the 33rd USENIX Security Symposium (Aug 14-16 2024)*.
- [10] ZHANG, Y., CHEN, X., DONG, Y., ZHANG, Q., HOU, R., LIU, Q., and CHEN, X. 2025. MD-SONIC: Maliciously-Secure Outsourcing Neural Network Inference With Reduced Online Communication. *Ieee Transactions on Information Forensics and Security* 20, 3534-3549.
- [11] BI, R.W., XIONG, J.B., LUO, C.Q., NING, J.T., LIU, X.M., TIAN, Y.L., and ZHANG, Y. 2024. Communication-Efficient Privacy-Preserving Neural Network Inference via Arithmetic Secret Sharing. *Ieee Transactions on Information Forensics and Security* 19, 6722-6737.
- [12] LI, J.G., YAN, Y., ZHANG, K., LI, C.L., and YUAN, P.C. 2024. FPCNN: A fast privacy-preserving outsourced convolutional neural network with low-bandwidth. *Knowledge-Based Systems* 283(Jan).
- [13] LIU, X.N., ZHENG, Y.F., YUAN, X.L., and YI, X. 2023. Securely Outsourcing Neural Network Inference to the Cloud With Lightweight Techniques. *Ieee Transactions on Dependable and Secure Computing* 20, 1 (Jan), 620-636.
- [14] KOTI, N., PANCHOLI, M., PATRA, A., SURESH, A., and ASSOC, U. 2021. SWIFT: Super-fast and Robust Privacy-Preserving Machine Learning. In *30th USENIX Security Symposium*, Electr Network, 2651-2668.
- [15] CHENG, K., XI, N., LIU, X., ZHU, X., GAO, H., ZHANG, Z., and SHEN, Y. 2023. Private Inference for Deep Neural Networks: A Secure, Adaptive, and Efficient Realization. *IEEE Transactions on Computers* 72, 12, 3519-3531.
- [16] DEMMLER, D., SCHNEIDER, T., and ZOHNER, M. 2015. ABY - A Framework for Efficient Mixed-Protocol Secure Two-Party Computation. In *Proceedings of the 22nd Annual Network and Distributed System Security Symposium (NDSS)* (San Diego, CA2015).
- [17] BOYLE, E., CHANDRAN, N., GILBOA, N., GUPTA, D., ISHAI, Y., KUMAR, N., and RATHEE, M. 2021. Function Secret Sharing for Mixed-Mode and Fixed-Point Secure Computation. In *Advances in Cryptology – EUROCRYPT 2021*, A. CANTEAUT and F.-X. STANDAERT Eds. Springer International Publishing, Cham, 871-900.

- [18] RYFFEL, T., THOLONIAT, P., POINTCHEVAL, D., and BACH, F.R. 2022. AriaNN: Low-Interaction Privacy-Preserving Deep Learning via Function Secret Sharing. In *Proceedings of the Privacy Enhancing Technologies (2022)*, 291-316.
- [19] HAO, M., LI, H., CHEN, H., XING, P., and ZHANG, T. 2023. FastSecNet: An Efficient Cryptographic Framework for Private Neural Network Inference. *IEEE Transactions on Information Forensics and Security* 18, 2569-2582.
- [20] BOYLE, E., GILBOA, N., and ISHAI, Y. 2016. Function Secret Sharing: Improvements and Extensions. In *Proceedings of the 23rd ACM Conference on Computer and Communications Security (CCS)* (Vienna, AUSTRIA, Oct 24-28 2016), 1292-1303.
- [21] RATHEE, D., RATHEE, M., KUMAR, N., CHANDRAN, N., GUPTA, D., RASTOGI, A., and SHARMA, R. 2020. CrypTFlow2: Practical 2-Party Secure Inference. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (ACM CCS)* (Nov 09-13 2020), 325-342.
- [22] RYFFEL, T., TRASK, A., DAHL, M., WAGNER, B., MANCUSO, J., RUECKERT, D., and PASSERAT-PALMBACH, J. 2018. A generic framework for privacy preserving deep learning. *CoRR abs/1811.04017*.
- [23] LECUN, Y., CORTES, C., and BURGESS, C.J. 2010. MNIST handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist> 2(2010).
- [24] KRIZHEVSKY, A. 2009. Learning multiple layers of features from tiny images.